Chiffrement ChaCha20

Conception d'un Système Numérique — Modélisation SystemVerilog

Auteur : Clément GREVEN

21 mai 2026

Table des matières

1 Introduction **2**

1.1 Contexte : le chiffrement symétrique par flux . . . 2

1.2 L'algorithme ChaCha20 . . . 2

1.3 Objectif du projet . . . 3

1.4 Outils et formalisme . . . 3

1.4.1 Outils utilisés . . . 3

1.4.2 Modules fournis . . . 3

2 Architecture de l'algorithme ChaCha20 **3**

2.1 L'état courant et sa représentation . . . 3

2.2 Vue d'ensemble de l'algorithme . . . 4

3 Module ARX **4**

3.1 Explication théorique . . . 5

3.2 Simulation et validation . . . 5

4 Le QuarterRound **6**

4.1 Explication théorique . . . 6

4.2 Simulation et validation . . . 7

5 Rondes colonne et rotation de lignes **7**

5.1 La ronde en colonne (`round_colonne`) . . . 8

5.2 Simulation des rondes diagonales par rotation de lignes . . . 8

5.2.1 Multiplexeur d'initialisation . . . 9

5.2.2 Simulation et validation . . . 9

6 Bloc ChaCha20 (`chacha_bloc`) **10**

6.1 Architecture interne . . . 10

6.2 Interface du module . . . 11

6.3 Simulation et validation . . . 11

7 Machine d'états finis (FSM) **12**

7.1 Présentation générale . . . 12

7.2 Graphe des états 13
7.3 Table des sorties 14
7.4 Simulation et validation 14

8 Module top-level (`chacha20_top`) 14
8.1 Architecture globale 15
8.2 Interface du module 15
8.3 Compteur de rondes (`compteur_init`) 16
8.4 Module XOR (`xor_module`) 16
8.5 Registre de keystream (`reg512`) 16
8.6 Intégration et validation 16

9 Problèmes rencontrés et solutions apportées 18
9.1 Difficultés rencontrées au cours du développement 18
9.2 Limites de l'implémentation et améliorations envisageables 18

10 Conclusion 18

1. Introduction

1.1 Contexte : le chiffrement symétrique par flux

L'histoire de la cryptographie, du grec *kryptós* (« caché ») et *graphein* (« écrire »), remonte à l'Antiquité et au Code de César avec la volonté de protéger l'information. Initialement développée pour chiffrer des messages diplomatiques, militaires, ou personnels, la cryptographie a évolué au fil du temps, passant de simples chiffres manuels à des algorithmes numériques sophistiqués.

Cette évolution est la conséquence de la lutte constante entre le développement de nouvelles méthodes de chiffrement et les efforts des cryptanalystes pour les briser.

La crytographie visait initiallement à la confidentialité, son implémentation dans l'informatique moderne a élargi son rôle. Aujourd'hui, elle permet la sécurité globale des systèmes en garantissant quatre propriétés essentielles :

- la **confidentialité**, elle garantit que seul le destinataire présumé peut lire le message.
- l'**intégrité** garantit que le message est original (aucune modification accidentelle ou malveillante a eu lieu durant le transport).
- l'**authentification** permet de confirmer l'identité de l'émetteur.
- la **non-répudiation** garantit que l'auteur ne peut pas contester le fait d'avoir émit le message

Ces quatre piliers reposent sur deux grandes familles d'algorithmes : les algorithmes à chiffrement **symétrique**, comme l'AES, très utilisés dans le stockage (comme BitLocker sur Windows 11), les flux de données (VPN, etc.) tandis que les algorithmes à chiffrement **asymétrique**, comme le RSA, sont très utilisés lors des échanges de clés (souvent avant de communiquer avec l'AES) et pour les signatures électroniques.

1.2 L'algorithme ChaCha20

ChaCha20 est, tout comme l'AES, un algorithme de chiffrement symétrique. Il a été développé par le mathématicien cryptologue et informaticien américain Daniel J. Bernstein, et standardisé dans la RFC 7539.

Le fonctionnement de ChaCha20 repose sur l'utilisation d'une clé unique et d'un nonce pour garantir que chaque message est chiffré différemment.

ChaCha20 se distingue de l'AES par plusieurs aspects. L'AES repose sur des opérations de substitution et de permutation sur des tableaux d'octets (ce qui le rend vulnérable

aux attaques par canaux cachés sur certaines implémentations matérielles, souvent sur la S-box), tandis que ChaCha20 effectue uniquement des additions, des rotations et des XOR. Ces opérations sont résistantes à ce type d'attaque.

De plus, ChaCha20 offre des performances très élevées, raison pour laquelle Google l'a adopté pour Chrome et Android ; il y trouve de surcroît une bonne efficacité énergétique.

1.3 Objectif du projet

Ce projet a pour objectif la modélisation matérielle de l'algorithme de chiffrement ChaCha20 en langage SystemVerilog. Chaque fonction est développée indépendamment, puis validée par un testbench dédié avant d'être intégrée dans l'architecture globale.

1.4 Outils et formalisme

1.4.1 Outils utilisés

Le développement a été réalisé en **SystemVerilog** à l'aide de l'environnement de simulation **ModelSim**.

1.4.2 Modules fournis

Durant ce projet, j'ai utilisé plusieurs modules fournis que j'importe afin de simplifier mon code.

Le package `chacha_pck.sv` définit le type `state_t`, représentant l'état interne sous la forme d'un tableau de 16 mots de 32 bits (512 bits au total).

Le module `rotation_pck.sv` définit le type `rotation_t` contenant `COL2DIAG` et `DIAG2COL`, un signal permettant la bonne réalisation des rondes (cf. partie 5).

2. Architecture de l'algorithme ChaCha20

2.1 L'état courant et sa représentation

L'algorithme ChaCha20 opère sur un état courant de 512 bits, organisé en une matrice 4×4 de mots de 32 bits. L'algorithme applique 20 rondes de transformations dont 10 en colonne et 10 en diagonale, puis calcule le keystream par addition modulo 2^{32} entre l'état final et l'état initial.

Le chiffrement est obtenu par XOR entre le keystream et le texte clair.

$$S = \begin{pmatrix} s_0 & s_1 & s_2 & s_3 \\ s_4 & s_5 & s_6 & s_7 \\ s_8 & s_9 & s_{10} & s_{11} \\ s_{12} & s_{13} & s_{14} & s_{15} \end{pmatrix}$$

L'état initial S_{init} est construit à partir de quatre éléments :

Ligne	Contenu	Valeur
0	Constante "expand 32-byte k"	`0x61707865...0x6b206574`
1–2	Clé secrète (256 bits)	$k_0 \dots k_7$
3	Compteur de bloc + Nonce	`0x00000001`, n_0, n_1, n_2

En SystemVerilog, cet état est représenté par le type `state_t` :

```
package chacha_pck;
  typedef logic [0:15][31:0] state_t;
endpackage
```

Listing 1 – Type `state_t` dans `chacha_pck.sv`

2.2 Vue d'ensemble de l'algorithme

1. **Initialisation** : construction de S_{init} à partir de la constante, de la clé, du nonce et du compteur de bloc.
2. **20 rondes** : On applique de façon alternée 10 rondes en colonne et 10 rondes en diagonale, chacune composée de 4 QuarterRounds.
3. **Génération du keystream** : keystream $= S_{\text{final}} + S_{\text{init}} \pmod{2^{32}}$
4. **Chiffrement** : $C_i = P_i \oplus \text{keystream}[128i - 1 : 128(i-1)]$

3. Module ARX

Le module ARX (Addition-Rotation-XOR) est la brique primitive de ChaCha20. Il s'agit d'un circuit purement **combinatoire** qui prend en entrée trois mots de 32 bits et produit deux mots de 32 bits en sortie.

3.1 Explication théorique

La structure ARX est fréquemment utilisée en cryptographie symétrique, elle combine trois opérations élémentaires sur des mots de 32 bits : une addition modulo 2^{32} entre `augend_i` et `addend_i`, un ou-exclusif bit à bit entre `xor_i` et la somme obtenue, puis une rotation circulaire vers la gauche de r bits.

$$a \leftarrow a + b \pmod{2^{32}} \tag{1}$$

$$d \leftarrow d \oplus a \tag{2}$$

$$d \leftarrow d \lll r \tag{3}$$

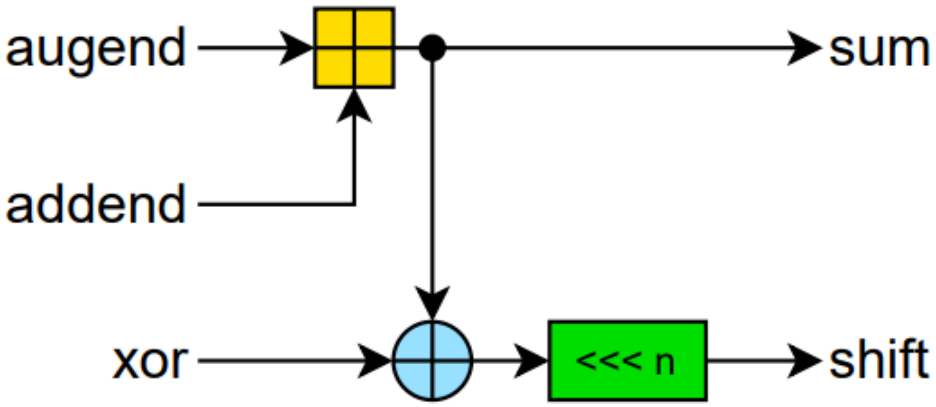


FIGURE 1 – Bloc ARX

La valeur de la rotation r dépend de la position du bloc ARX dans la structure QuarterRound que nous allons ensuite étudier.

3.2 Simulation et validation

Les résultats obtenus suite à une simulation sur ModelSim correspondent aux vecteurs de tests fournis dans le sujet. Cela confirme le bon fonctionnement du module ARX.

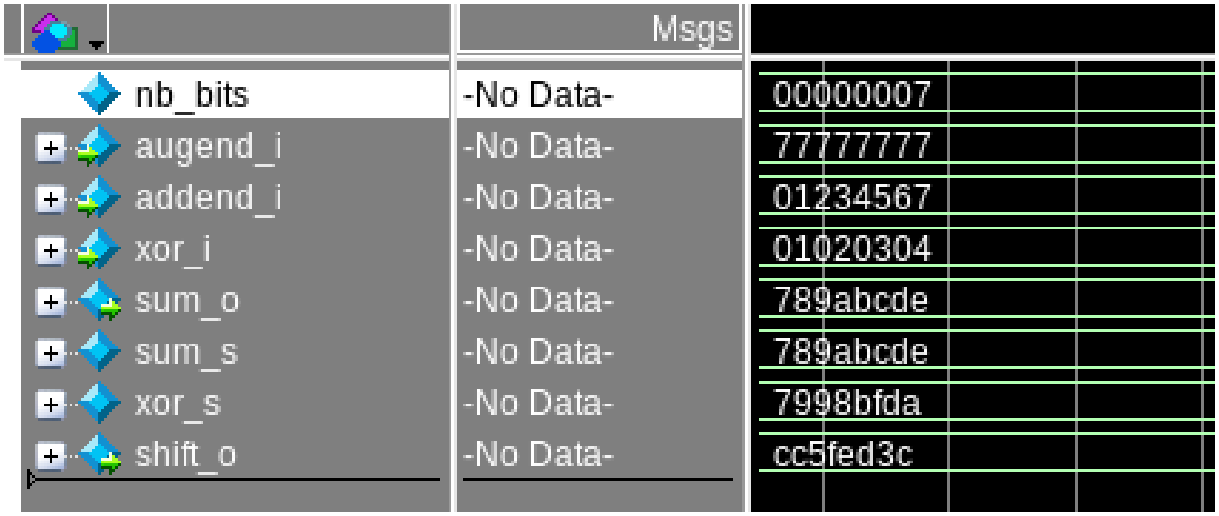


FIGURE 2 – Simulation du module `arx` — vecteurs de test du sujet.

4. Le QuarterRound

Le QuarterRound prend en entrée quatre mots de 32 bits (notés a, b, c, d) et produit quatre nouveaux mots en sortie après l'application successive de quatre blocs ARX avec des rotations différentes (16, 12, 8 et 7 bits). Ce module est, lui aussi, purement **combinatoire**.

4.1 Explication théorique

$$a \leftarrow a + b; \quad d \leftarrow (d \oplus a) \lll 16 \tag{4}$$

$$c \leftarrow c + d; \quad b \leftarrow (b \oplus c) \lll 12 \tag{5}$$

$$a \leftarrow a + b; \quad d \leftarrow (d \oplus a) \lll 8 \tag{6}$$

$$c \leftarrow c + d; \quad b \leftarrow (b \oplus c) \lll 7 \tag{7}$$

Pour la modélisation, je définis quatre signaux internes afin de relier les blocs ARX. Le premier, `n1_s` est la sortie du premier ARX (rotation 16 bits), `n2_s` celle du deuxième (rotation 12 bits), `n3_s` celle du troisième (rotation 8 bits) et `n4_s` la sortie finale du quatrième ARX (rotation 7 bits).

Le QuarterRound contient uniquement des blocs combinatoires, en cela, il est un module purement combinatoire.

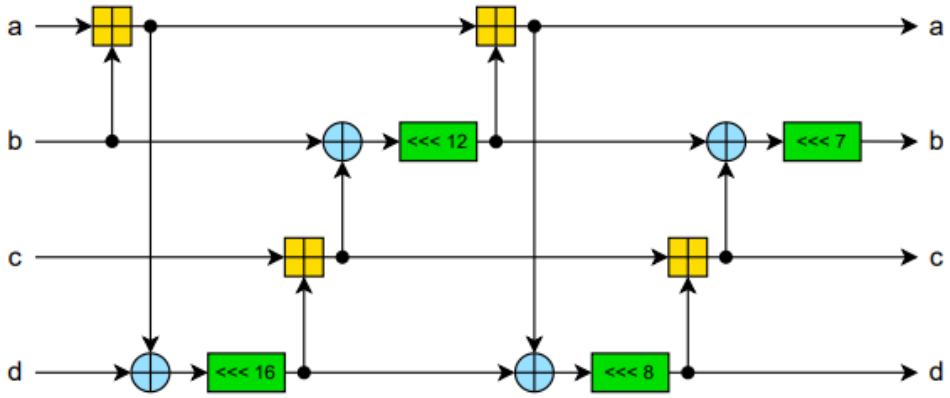


FIGURE 3 – Bloc `QuarterRound` .

4.2 Simulation et validation

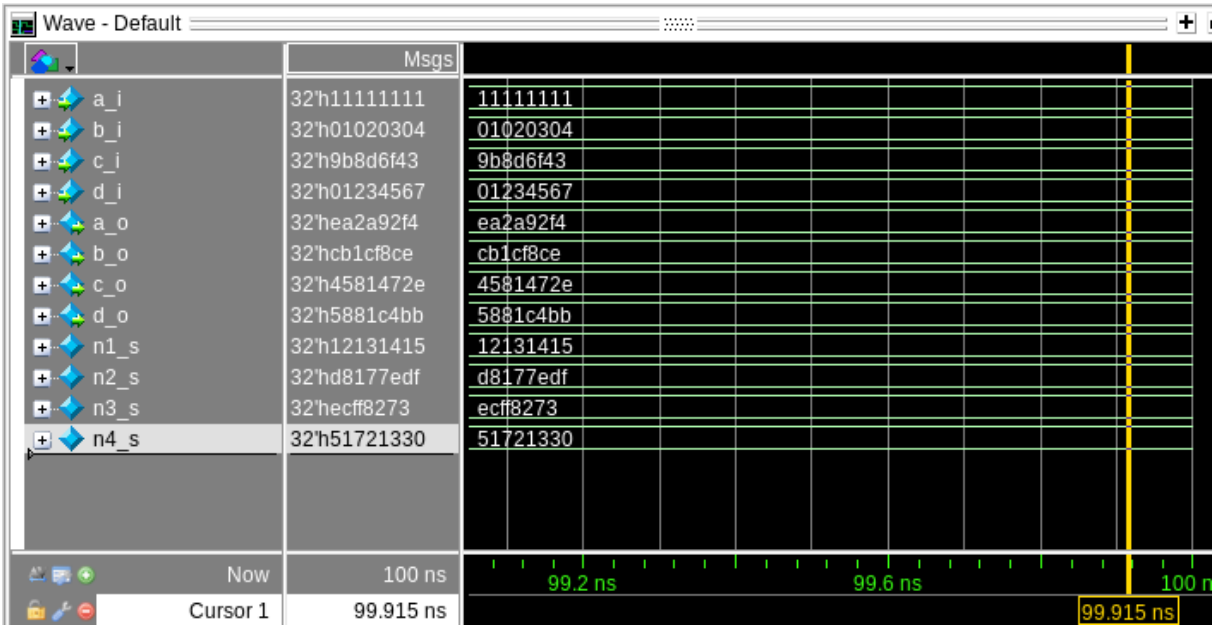


FIGURE 4 – Simulation du module `QuarterRound` .

Les résultats obtenus suite à une simulation sur ModelSim correspondent aux vecteurs de tests fournis dans le sujet. Cela confirme le bon fonctionnement du module QuarterRound.

5. Rondes colonne et rotation de lignes

L'algorithme ChaCha20 alterne, sur 20 rondes au total, une ronde en colonne et une ronde en diagonale. Chaque ronde consiste en l'application simultanée de quatre QuarterRounds portant sur des groupes de quatre mots distincts extraits de la matrice d'état 4×4.

5.1 La ronde en colonne (`round_colonne`)

Puisque l'état interne de ChaCha20 est composé de 16 mots de 32 bits, il est nécessaire d'appliquer quatre fois la fonction QuarterRound.

Les rondes en colonne s'appliquent donc de la façon suivante :

$$\mathrm{QR}(s_0,\ s_4,\ s_8,\ s_{12})$$
$$\mathrm{QR}(s_1,\ s_5,\ s_9,\ s_{13})$$
$$\mathrm{QR}(s_2,\ s_6,\ s_{10},\ s_{14})$$
$$\mathrm{QR}(s_3,\ s_7,\ s_{11},\ s_{15})$$

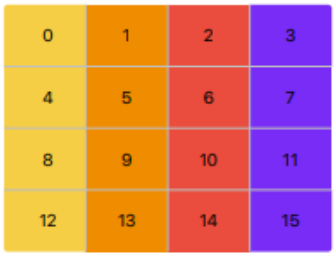


FIGURE 5 – Matrice en colonne

5.2 Simulation des rondes diagonales par rotation de lignes

Nous venons d'étudier la réalisation des rondes en colonne. Cependant, ChaCha20 est fait de rondes en colonnes mais aussi de rondes en diagonales.

Les rondes en diagonale s'appliquent de cette façon :

$$\mathrm{QR}(s_0,\ s_5,\ s_{10},\ s_{15})$$
$$\mathrm{QR}(s_1,\ s_6,\ s_{11},\ s_{12})$$
$$\mathrm{QR}(s_2,\ s_7,\ s_8,\ s_{13})$$
$$\mathrm{QR}(s_3,\ s_4,\ s_9,\ s_{14})$$

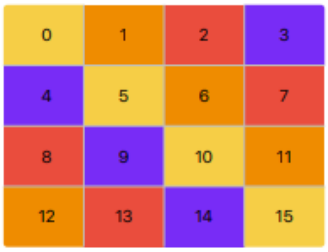


FIGURE 6 – Matrice en diagonale

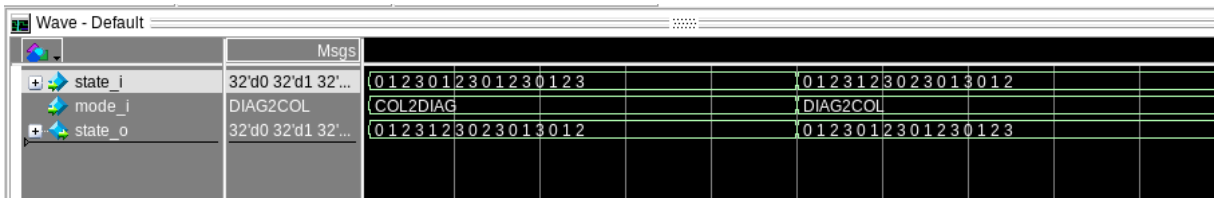


FIGURE 7 – Simulation du module `rotation`

Plutôt que de créer un module de ronde diagonale dédié, je réalise la ronde diagonale en réutilisant `round_colonne`, précédé d'une permutation des lignes par le module `rotation`.

Le module `rotation` fonctionne à l'aide d'une variable de type `rotation_t` définie dans le package `rotation_pck.sv`. Ce type contient deux valeurs de type logique `COL2DIAG` et `DIAG2COL` qui indiquent de quelle façon transformer la matrice en vue de la prochaine ronde.

Afin de tester le fonctionnement de ma fonction rotation, j'initialise l'état d'entrée avec des valeurs distinctes selon leur colonne (0, 1, 2, 3). Je leur applique d'abord la transformation `COL2DIAG` pour commencer et je vérifie que les cases sont bien décalées, puis j'applique ensuite la transformation `DIAG2COL` sur le résultat obtenu. Les deux transformations étant inverses, on retrouve bien l'état initial. (COL2DIAG $\times$ DIAG2COL = identité)

Note : afin d'afficher tous les bits des signaux `state_i` et `state_o` sur l'écran wave, j'ai changé leur mode d'affichage en base décimale.

5.2.1 Multiplexeur d'initialisation

Dans le module `chacha_bloc`, un multiplexeur sélectionne l'état qui sera traité par le bloc `round_colonne`. Lors du premier cycle (`init_i` = 1), le multiplexeur sélectionne l'état initial `init_state_i` afin de l'injecter dans la première ronde. Pour les cycles suivants (`init_i` = 0), il sélectionne la sortie qui représente le résultat de la ronde précédente.

5.2.2 Simulation et validation

Les résultats obtenus suite à une simulation correspondent aux vecteurs de tests fournis. Le testbench commence par initialiser le `keystream_i` à zéro, puis donne une assigne des valeurs reconnaissable afin de les identifier sur modelsim. Le compteur est ensuite incrémenté et je remarque que les

Le testbench initialise le `keystream_i` à zéro, puis force `keystream_i[0]` = `0x00000001`

et `keystream_i[4]` = `0x00000002` afin d'identifier facilement quel quart est sélectionné. Le compteur `cpt_i` est ensuite balayé sur ses quatre valeurs possibles avec un délai de 50 ns entre chaque changement. Cela confirme le bon fonctionnement du module.

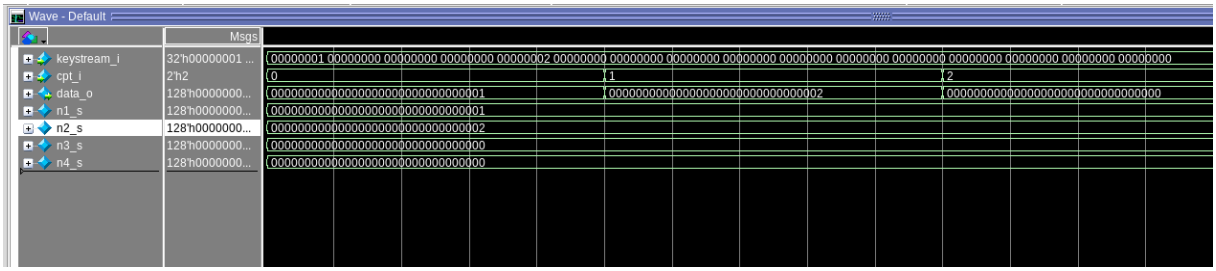


FIGURE 8 – Simulation du multiplexeur d'initialisation dans `chacha_bloc`.

6. Bloc ChaCha20 (`chacha_bloc`)

Ce module utilise les fonctions définies précédemment afin de réaliser 20 rondes de transformation (10 en colonnes et 10 en diagonales) avec un état interne de 512 bits.

Un signal `init_i` permet de charger l'état initial et `reg_sel_i` sélectionne si l'on fourni au registre le résultat de la ronde ou l'état initial. Après les 20 rondes, le keystream est généré à partir d'une addition (modulo 2^{32}) entre l'état final et l'état initial.

Le module `chacha_bloc` intègre un registre d'état (`registre_d`) qui mémorise le résultat de chaque ronde et le fournit en entrée de la ronde suivante. Ce mécanisme rend le module **séquentiel**.

6.1 Architecture interne

Le bloc ChaCha20 importe les modules définis précédemment (`round_colonne`, `registre_d`, `rotation`, `additionneur`) afin de réaliser 20 rondes dont 10 en colonnes et 10 en diagonales. Suite à cela, l'additionneur réalise une addition modulo 2^{32} entre la sortie des 20 rondes et l'état initial.

Durant l'exécution du ChaChabloc, un multiplexeur en entrée sélectionne l'état initial pour le charger dans la première ronde ou l'état du registre pour le calcul des rondes suivantes.

À l'intérieur des rondes, le bloc `round_colonne` applique quatre QuarterRounds, `rotation` permute les cases de la matrice, puis Reg State mémorise l'état courant.

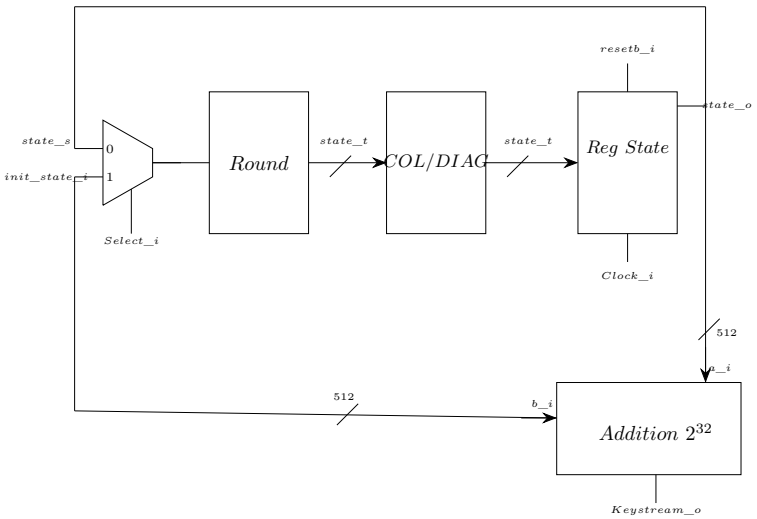


FIGURE 9 – Architecture interne du module `chacha_bloc`.

6.2 Interface du module

```
module chacha_bloc
import chacha_pck::*;
import rotation_pck::*;
(
  input  logic      clock_i,
  input  logic      resetb_i,
  input  state_t    init_state_i, // etat initial S_init
  input  logic      init_i,       // 1 = premier cycle (injecter S_init)
  input  rotation_t rotation_i,   // COL2DIAG ou DIAG2COL
  input  logic      reg_sel_i,    // enable registre d'etat
  output state_t    keystream_o   // keystream apres addition (512 bits)
);
```

Listing 2 – Interface du module `chacha_bloc`

6.3 Simulation et validation

Afin de tester le bon fonctionnement du bloc ChaCha20, je réalise un testbench dans lequel j'initialise l'entrée à partir de celle fournie à la fin du sujet. Je réalise 20 rondes en alternant les rondes en colonne et en diagonale puis je constate le fait que la valeur obtenue en sortie correspond à la valeur attendue à la fin du sujet.

Je ne suis pas parvenu à obtenir la même valeur et n'ai malheureusement pas eu le temps de proprement déboguer.

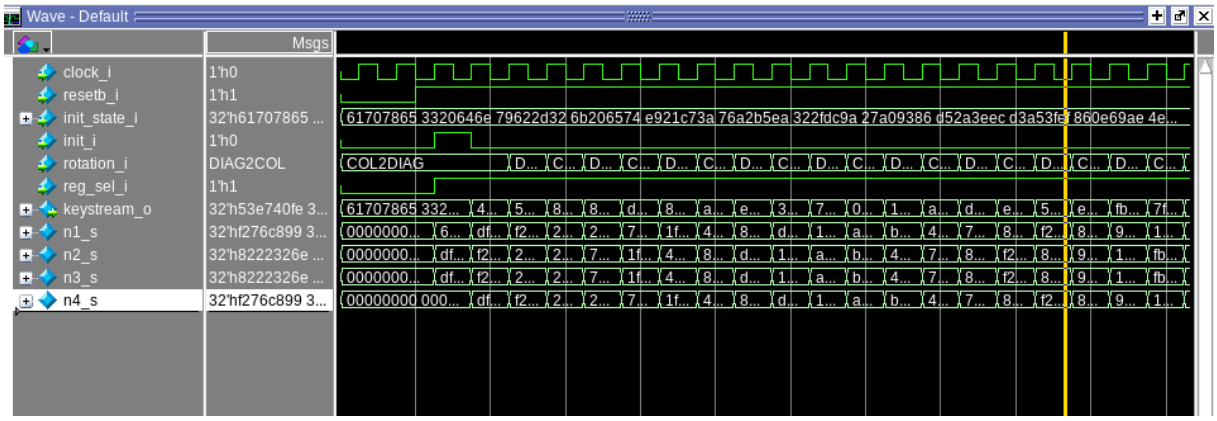


FIGURE 10 – Simulation du module `chacha_bloc`.

7. Machine d'états finis (FSM)

La machine d'états finis (FSM) séquence les opérations du chiffrement en générant les signaux de contrôle à destination des autres modules (`chacha_bloc`, compteur, registre de sortie, module XOR).

7.1 Présentation générale

Il s'agit d'une machine de Moore, c'est-à-dire que les sorties ne dépendent uniquement de l'état courant. Cependant, trois sorties — `init_o`, `rotation_o` et `xor_cpt_o` — dépendent également du signal `cpt_i` en plus de l'état courant, introduisant un comportement de Mealy partiel pour ces signaux.

7.2 Graphe des états

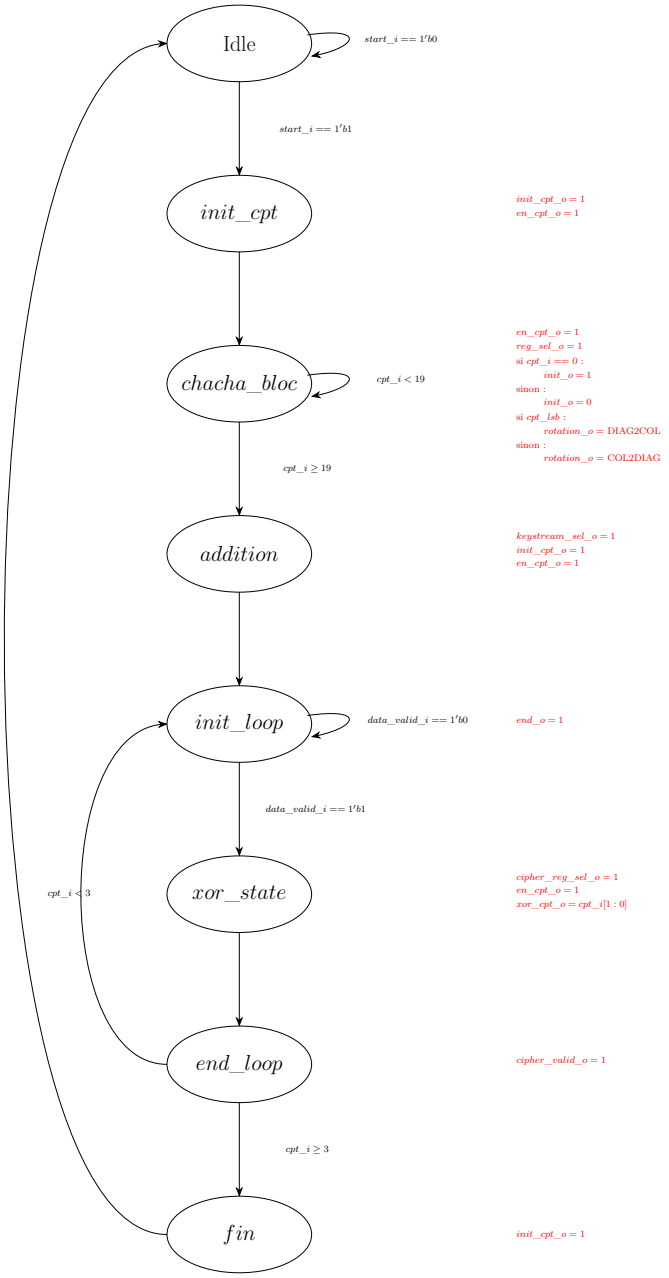


FIGURE 11 – Graphe des états de la FSM `chacha20_top`.

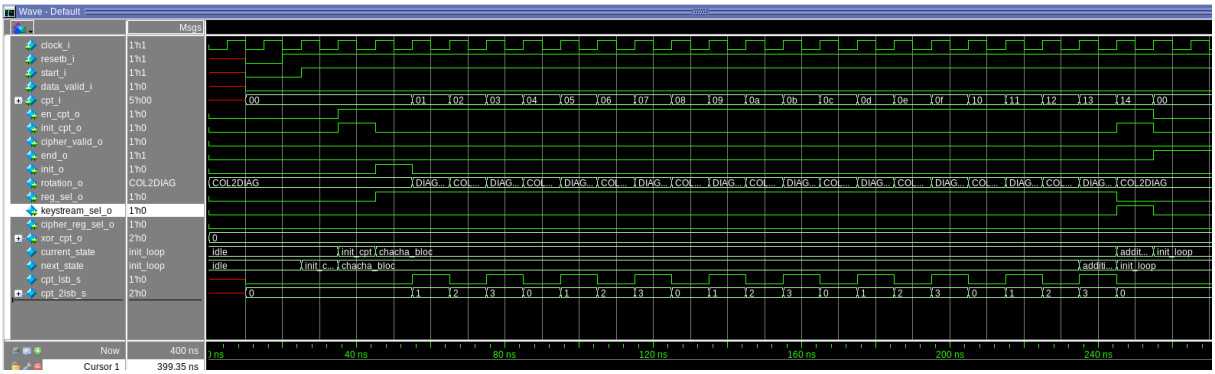


FIGURE 12 – Simulation de la FSM — enchaînement des états et signaux de contrôle.

7.3 Table des sorties

Signal	*idle*	*init_cpt*	*chacha*	*addition*	*init_loop*	*xor*	*end_loop*	*fin*
`en_cpt_o`	0	1	1	1	0	1	0	0
`init_cpt_o`	0	1	0	1	0	0	0	1
`init_o`	0	0	$\star$	0	0	0	0	0
`reg_sel_o`	0	0	1	0	0	0	0	0
`keystream_sel_o`	0	0	0	1	0	0	0	0
`cipher_reg_sel_o`	0	0	0	0	0	1	0	0
`cipher_valid_o`	0	0	0	0	0	0	1	0
`end_o`	0	0	0	0	1	0	0	0

$\star$: `init_o=1` uniquement lorsque `cpt_i=0`.

7.4 Simulation et validation

Un testbench instancie la FSM et le compteur. Le signal `start_i` est activé afin de vérifier le bon fonctionnement de la FSM, puis le signal reset est activé et relâché afin que la FSM commence dans son premier état.

8. Module top-level (`chacha20_top`)

Le module `chacha20_top` est le plus haut niveau d'intégration. Il instancie et interconnecte l'ensemble des modules développés au long du projet dans l'objectif de former le circuit.

Concrètement, ce module construit l'état initial à partir de la constante, de la clé et du nonce, le fournit au bloc `chacha_bloc`, puis récupère le keystream produit après les 20 rondes. Il intègre également le compteur de rondes qui indique à la machine d'état

le numéro de la ronde courante, le module XOR qui chiffre chaque bloc de 16 octets du message en appliquant un XOR avec la tranche correspondante du keystream, et un registre de 512 bits qui mémorise le keystream.

8.1 Architecture globale

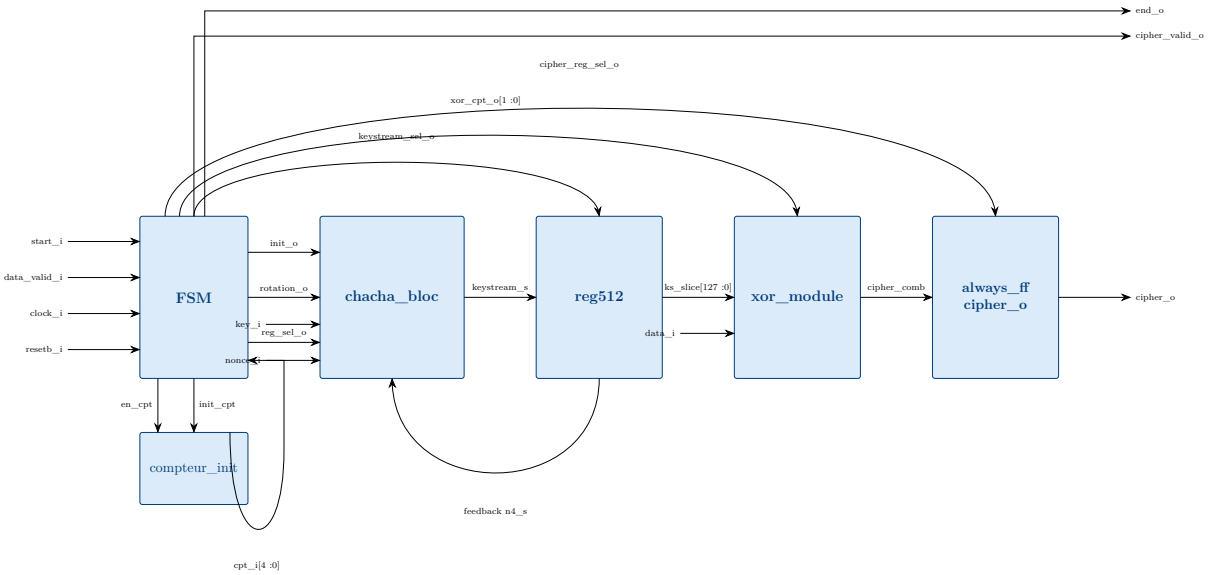


FIGURE 13 – Architecture globale du module `chacha20_top`.

8.2 Interface du module

```
module chacha20_top
import chacha_pck::*;
import rotation_pck::*;
(
  input  logic          clock_i,
  input  logic          resetb_i,
  input  logic          start_i,        // demarre le chiffrement
  input  logic [127:0]  data_i,         // texte clair (128 bits)
  input  logic          data_valid_i,   // donnee disponible
  input  logic [255:0]  key_i,          // cle secrete (256 bits)
  input  logic [95:0]   nonce_i,        // nonce (96 bits)
  output logic [127:0]  cipher_o,       // texte chiffre (128 bits)
  output logic          cipher_valid_o, // bloc chiffre disponible
  output logic          end_o           // FSM prete a recevoir du texte
);
```

Listing 3 – Interface du module `chacha20_top`

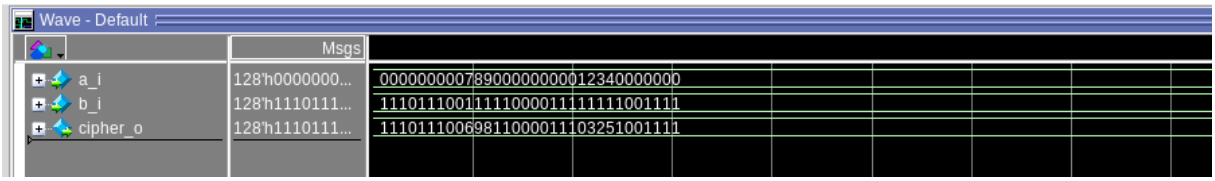


FIGURE 14 – Simulation du module XOR

8.3 Compteur de rondes (`compteur_init`)

Le module fonctionne de la manière suivante : lorsque `resetb_i=0`, le compteur est réinitialisé à 0. Lorsque `resetb_i=1` et que le write enable `en_i` est actif, si `init_i=1`, le compteur est remis à 0. Sinon, le compteur s'incrémente de 1 à chaque front montant d'horloge.

Le compteur renvoie une valeur sur 5 bits, ce qui permet de compter jusqu'à 31. Cela est suffisant pour les deux usages : les 20 rondes ChaCha (valeurs jusque 19) et les blocs XOR.

Ce module étant fourni et déjà validé, aucun testbench dédié n'a été réalisé.

8.4 Module XOR (`xor_module`)

Le module `xor_module` effectue un ou-exclusif bit à bit sur 128 bits entre une tranche du keystream et le bloc de texte clair `data_i`, produisant le bloc chiffré `cipher_o`.

Afin de tester notre module XOR, il suffit de rentrer deux mots en entrée et de vérifier que la sortie correspond bien à l'application bit-à-bit d'un ou-exclusif.

8.5 Registre de keystream (`reg512`)

Ce registre est utile à la mémorisation du keystream suite à l'exécution des 20 rondes. Ainsi, on manipule des données de type `state_t`, la matrice de 16 cases de 32 bits.

Tout comme pour le module compteur, on se sert d'un reset actif bas. Lorsque le reset est inactif, à chaque front montant d'horloge et uniquement si `sel_i=1`, la sortie `state_o` prend la valeur de l'entrée `state_i`.

8.6 Intégration et validation

Je construis l'état initial à partir des vecteurs de tests fournies dans l'énoncé.

Durant l'exécution, la machine d'état permet l'alternance des rondes en colonnes et en diagonales à partir du signal interne `rotation_s`. Je vérifie sur le chronogramme que ce signal bascule parfaitement entre `COL2DIAG` et `DIAG2COL` à chaque cycle, enchaînant ainsi les 10 cycles complets (soit les 20 rondes cumulées suivies par le compteur `cpt_s`). La

bonne exécution du chemin de données est validée à la fin du calcul lorsque le signal `end_o` passe à 1 et que le résultat final obtenu sur la sortie `cipher_o` concorde parfaitement avec la valeur de chiffrement théorique attendue.

Je vérifie la présence du signal `cipher_valid_o` délivré par ma machine d'état.

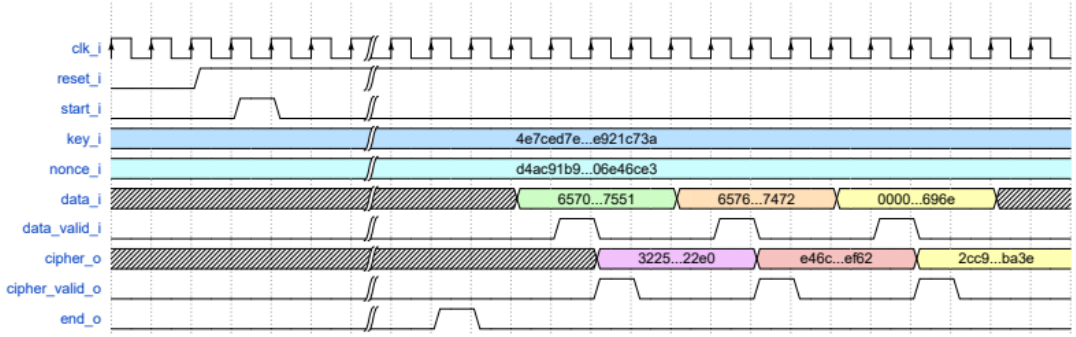


FIGURE 15 – Fonctionnement interne de l'algorithme.

Le signal `data_valid_i` annonce que la donnée `data_i` est valide et peut être utilisée en vue du chiffrement. Le signal `cipher_valid_o` indique que les données chiffrées sont valides. Il est activé dans l'état ***`end_loop`*** de ma machine d'état, un cycle après le XOR.

Afin de valider le fonctionnement de `chacha20_top`, un testbench nous a été fourni. Ce testbench initialise les entrées (clé, nonce, ...), puis présente successivement les trois blocs du message clair via `data_i` en activant `data_valid_i` à chaque fois que `end_o` passe à 1.

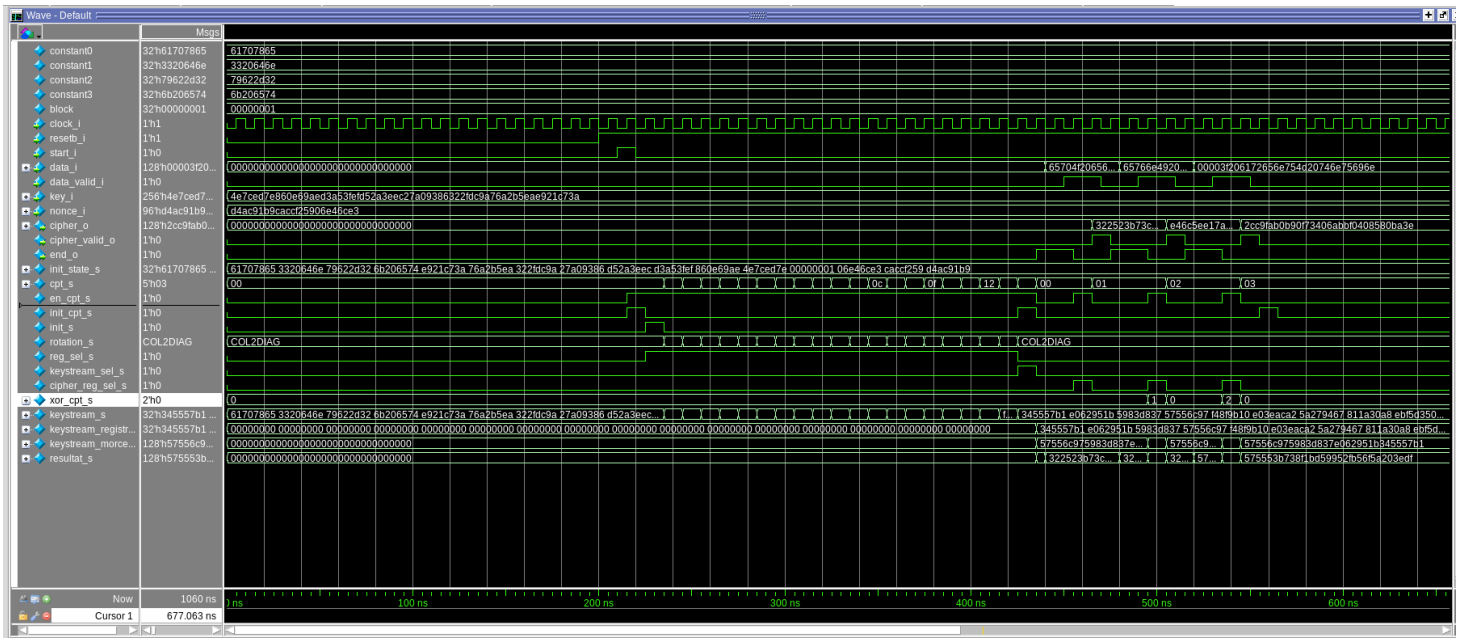


FIGURE 16 – Simulation complète du module sous ModelSim.

```
VSIM 3> run -all
# Ciphertext word #1 @470.000 ns: 322523b73ceabe5e8e05fc68143022e0
# Ciphertext word #2 @510.000 ns: e46c5ee17a42f515954acdecd4eeef62
# Ciphertext word #3 @550.000 ns: 2cc9fab0b90f73406abbf0408580ba3e
# Computed Ciphertext: fab0b90f73406abbf0408580ba3ee46c5ee17a42f515954acdecd4eeef62322523b73ceabe5e8e05fc68143022e0
# Expected Ciphertext: fab0b90f73406abbf0408580ba3ee46c5ee17a42f515954acdecd4eeef62322523b73ceabe5e8e05fc68143022e0
# Computed ^ Expected: 00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
# Test PASSED: Computed ciphertext matches expected value.
# ** Note: $finish    : ./SRC/BENCH/chacha20_top_tb.sv(123)
#    Time: 560 ns  Iteration: 0  Instance: /chacha20_top_tb
# 1
# Break in Module chacha20_top_tb at ./SRC/BENCH/chacha20_top_tb.sv line 123
```

FIGURE 17 – Validation de la simulation complète du module

On observe ainsi sur le chronogramme (*waveform*) de ModelSim trois impulsions successives du signal `cipher_valid_o`, ce qui indique la disponibilité et la bonne correspondance des messages chiffrés par rapport aux vecteurs de test attendus.

Le testbench vérifie ensuite par comparaison avec un XOR que les valeurs obtenues par mon algorithme `cipher_o` correspondent aux vecteurs de chiffrements attendus par l'énoncé.

9. Problèmes rencontrés et solutions apportées

9.1 Difficultés rencontrées au cours du développement

Durant le développement, j'ai initialement rencontré de nombreuses difficultés de compréhension du fonctionnement global de l'algorithme. De plus, j'ai aussi eu du mal à prendre en main le langage SystemVerilog.

Avec l'aide de mon cours, j'ai pu réussir à terminer le projet à bien.

En effet, une fois remis dans le bain avec mon cours en tête, j'ai été beaucoup à l'aise avec la création et le débogage des modules.

9.2 Limites de l'implémentation et améliorations envisageables

Il serait probablement possible de réduire l'état de ma machine d'état en regroupant plusieurs états au sein d'un même cycle d'horloge. Cependant, cela pourrait entraîner une diminution de la fréquence d'horloge maximale du circuit.

10. Conclusion

De prime abord, j'ai trouvé ce projet très difficile, il ne s'est pour autant avéré simple. Il m'aura permis de découvrir le SystemVerilog, qui, contrairement au C ou au Python,

est un langage de description matérielle.

J'ai ressenti une difficulté croissante lors de l'avancement du projet. J'ai pu progresser à mon rythme et terminer le projet. Ce projet m'aura été très formateur sur la maîtrise du SystemVerilog.

De plus, j'ai grandement apprécié modéliser ce circuit, et contrairement à l'AES, Chacha20 ne possède pas de S-box, réduisant significativement le risque d'attaques matérielles telles que les attaques par canaux cachés.